

Artificial Intelligence in Software Supply-Chain Security and Vulnerability Detection: A Comprehensive Survey of Threats, Methodologies, and Trustworthy Deployment

1 Introduction

The modern software supply chain has undergone a paradigmatic shift, evolving from a static assembly of source code dependencies into a dynamic ecosystem integrating pre-trained machine learning models, automated code generation tools, and complex dependency graphs. This transformation has exponentially expanded the attack surface, introducing vulnerabilities that transcend traditional source code boundaries to encompass data integrity, model weights, and generative outputs [1]. The urgency of addressing these systemic risks is underscored by the rapid industrial adoption of generative artificial intelligence (GenAI) and third-party models, which has outpaced the development of corresponding security governance frameworks [2]. Consequently, this survey establishes a critical intersection between artificial intelligence technologies and supply-chain security, defining a dual scope: leveraging AI as a defensive mechanism for vulnerability detection and, conversely, securing AI artifacts themselves as potential targets within the supply chain.

The distinction between these two roles is fundamental yet often conflated in contemporary literature. On one hand, AI serves as a potent force multiplier for security operations, utilizing deep learning architectures to identify subtle vulnerability patterns in source code and binaries that evade rule-based static analysis [3]. On the other hand, AI components—ranging from training datasets to serialized model weights—have become primary vectors for supply-chain compromise. Adversaries now exploit these components through sophisticated techniques such as data poisoning, backdoor injection, and the distribution of trojanized models via public repositories like Hugging Face and PyPI [4; 5]. Recent empirical studies reveal that the integration of AI coding assistants can inadvertently propagate insecure coding practices, with developers often exhibiting unwarranted trust in AI-generated suggestions, thereby amplifying the risk of vulnerability introduction at the source [6; 7].

Compared to traditional software security, the challenges inherent in AI-enabled supply chains are distinct in their non-deterministic nature and reliance on probabilistic rather than logical correctness. Traditional vulnerability detection relies on deterministic parsing and symbolic execution, whereas AI security must account for adversarial perturbations, distribution shifts, and the opacity of black-box models [8]. For instance, attacks such as package hallucinations by large language models introduce a novel class of supply-chain risk where non-existent, potentially malicious dependencies are synthesized and recommended to developers, a phenomenon unseen in conventional package management [9]. Furthermore, the transitive nature of dependencies in AI frameworks means that a compromise in a foundational library can cascade through thousands of downstream applications, complicating attribution and remediation efforts [10; 11].

To address these multifaceted challenges, this survey bridges the gap between software engineering security practices and machine learning safety research. We synthesize existing literature to provide a holistic view of the threat landscape, ranging from clean-label poisoning in training data to evasion attacks against AI-driven detectors [12; 13]. By categorizing threats based on the lifecycle stage they affect and evaluating the efficacy of current defense mechanisms, including cryptographic attestation and neuro-symbolic verification, we aim to guide the development of resilient, trustworthy AI ecosystems. The subsequent sections of this paper are organized to first delineate the taxonomy of threats, then explore AI-driven detection methodologies, followed by strategies for securing the

AI supply chain, and finally, discuss automated remediation and the socio-technical barriers to adoption, ultimately offering a strategic roadmap for future research and standardization [14].

2 Taxonomy of Threats and Attack Vectors in Artificial Intelligence-Enabled Supply Chains

2.1 Data-Centric Poisoning and Integrity Compromises in Model Training

Data-centric poisoning represents a foundational threat vector within the AI software supply chain, wherein adversaries manipulate the integrity of training corpora to embed latent vulnerabilities, biases, or backdoors that persist through model deployment. Unlike traditional software exploits that target runtime logic, these attacks compromise the statistical learning process itself, often maintaining high accuracy on benign inputs while inducing specific erroneous behaviors under attacker-defined conditions. The spectrum of these threats ranges from overt label flipping to sophisticated clean-label poisoning, where malicious samples are injected with correct labels but optimized via gradient matching to distort the model’s decision boundary without triggering standard data validation checks [12]. Recent advancements have demonstrated that such poisoning strategies are particularly potent in neural code completion systems; by introducing a minimal number of crafted files into public repositories, attackers can “teach” models to suggest insecure cryptographic primitives or vulnerable API usage patterns, effectively scaling the propagation of security anti-patterns across the ecosystem [15].

A critical evolution in this domain is the shift towards stealthy, context-aware injection techniques designed to evade static analysis and dataset cleansing. Traditional poisoning often relies on explicit malicious payloads, which can be filtered by signature-based defenses. However, novel approaches such as TrojanPuzzle and Covert attacks circumvent these measures by planting malicious logic in out-of-context regions like docstrings or by fragmenting payloads such that no single training sample contains the complete exploit [16]. Furthermore, the emergence of large language models (LLMs) has introduced new surfaces for fine-tuning dataset contamination. Attacks like CodeBreaker leverage LLMs to transform malicious payloads into semantically equivalent but syntactically disguised variants, ensuring that both the poisoned fine-tuning data and the resulting generated code evade strong vulnerability detectors [17]. This capability highlights a dangerous feedback loop: as models are increasingly trained on AI-generated code, the potential for “un-repairing” clean code through targeted poisoning becomes a significant systemic risk [18].

Detecting these compromises remains a formidable challenge due to the high dimensionality of modern embedding spaces and the subtlety of the induced perturbations. While early research suggested that backdoors leave distinct spectral signatures in learned representations, enabling detection via robust statistical methods [19], adaptive attackers are now employing adversarial features to create triggers that bypass such forensic analysis. For instance, the AFRAIDDOOR framework utilizes adaptive adversarial perturbations to inject stealthy backdoors that achieve high success rates even against state-of-the-art defense mechanisms, rendering traditional spectral analysis insufficient [20]. Moreover, recent investigations into model parameters reveal that while activation values and context embeddings may exhibit anomalies in poisoned models, attention weights often remain indistinguishable from clean counterparts, complicating white-box inspection efforts [21]. Consequently, the field is moving towards more rigorous data provenance tracking and dynamic validation frameworks, such as DataBOMs, to mitigate the risks associated with unvetted third-party datasets. Future research must prioritize the development of robust learning algorithms capable of distinguishing between genuine semantic patterns and adversarial noise, alongside stan-

standardized benchmarks that accurately reflect the complexity of real-world supply chain poisoning scenarios [22; 23].

2.2 Model-Centric Threats and Malicious Artifact Distribution

Building upon the foundational vulnerabilities established in data-centric poisoning, model-centric threats represent a critical escalation in AI supply-chain security, shifting the attack surface from the training corpus to the model artifacts themselves. Whereas data poisoning requires infiltration of the training pipeline to embed latent biases, model-centric attacks treat pre-trained weights and architectures as mutable objects that can be directly manipulated post-training or embedded with malicious payloads prior to distribution. This paradigm enables adversaries to compromise downstream applications simply by introducing trojanized models into public repositories, exploiting the widespread practice of downloading and integrating third-party artifacts without rigorous integrity verification.

A primary vector in this domain is the handcrafted backdoor attack, where adversaries directly modify neural network weights to embed dormant triggers. [24] demonstrate that direct weight manipulation offers attackers significantly more degrees of freedom compared to traditional poisoning, allowing them to evade many existing detection and removal defenses while maintaining high attack success rates across diverse architectures. This approach is further refined in deployment-stage attacks, such as the Subnet Replacement Attack (SRA), which operates in a gray-box setting to inject backdoors into deployed models on user devices, effectively transforming the model file into a stealthy computer virus [25]. These techniques highlight a fundamental vulnerability: the assumption that model parameters are benign numerical values rather than executable logic capable of being hijacked.

Beyond behavioral manipulation, model artifacts serve as potent carriers for steganographic malware, extending the threat from functional compromise to systemic infection. [26] introduce Malefic-Net 2.0, a sophisticated technique that utilizes spread-spectrum channel coding and error correction to embed self-extracting, self-executing malware directly within model parameters. This method ensures that the malicious payload remains stealthy, does not degrade model performance, and survives quantization or pruning, posing a severe risk when models are shared via untrusted hubs. The danger is compounded by inherent vulnerabilities in common model serialization formats. Formats such as Python’s Pickle, widely used in frameworks like PyTorch, allow arbitrary code execution during the deserialization process. Consequently, loading an untrusted model file can instantly compromise the host system, bypassing traditional software security controls that focus on source code rather than binary tensor data.

The distribution of such malicious artifacts is facilitated by the lack of robust provenance tracking in model registries, a gap that parallels the data lineage challenges discussed previously but with immediate execution consequences. While traditional software supply chains rely on cryptographic signing and Software Bill of Materials (SBOM), the AI ecosystem often lacks equivalent mechanisms for verifying model lineage and weight integrity. This gap allows attackers to upload trojanized models to platforms like Hugging Face, where they may be downloaded and integrated by unsuspecting developers. Furthermore, the complexity of modern models makes manual inspection infeasible, and automated scanners often fail to detect weight-level perturbations or steganographic payloads that do not alter the model’s primary task accuracy.

Addressing these threats requires a paradigm shift towards treating model artifacts as untrusted executables, setting the stage for the broader systemic risks introduced by generative agents. Future

defenses must integrate cryptographic attestation of model weights, secure serialization formats that disable arbitrary code execution, and runtime monitoring capable of detecting anomalous weight activation patterns. As the reliance on pre-trained models grows, the security community must develop standardized protocols for model verification akin to those established for traditional software binaries, ensuring that the integrity of the AI supply chain is maintained from the repository to the runtime environment before these artifacts are leveraged by the generative tools discussed in the following section.

2.3 Generative AI Risks and Vulnerability Propagation in Code Synthesis

The integration of Large Language Models (LLMs) into the software development lifecycle has fundamentally altered the threat landscape, transforming generative tools from mere productivity enhancers into potential vectors for systemic vulnerability propagation. Unlike traditional supply chain risks rooted in explicit dependency management, generative AI introduces stochastic risks where models act as amplifiers, replicating insecure patterns found in training corpora at an unprecedented scale. Empirical studies indicate that state-of-the-art coding assistants, such as GitHub Copilot, generate vulnerable code in approximately 40% of scenarios, frequently suggesting insecure cryptographic modes or flawed input validation logic [27]. This phenomenon is not merely a failure of filtration but a reflection of the models’ memorization of prevalent anti-patterns within open-source repositories, creating a feedback loop where historical vulnerabilities are continuously reintroduced into modern codebases [15].

A critical and emerging vector within this domain is the hallucination of non-existent software dependencies, termed “package hallucinations.” When LLMs fabricate library names that syntactically resemble legitimate packages, they inadvertently facilitate typosquatting attacks, prompting developers to install malicious components from public registries. Research reveals that package hallucinations occur in at least 5.2% of commercial model outputs and up to 21.7% in open-source variants, representing a novel form of supply chain confusion that bypasses traditional static analysis [9]. Furthermore, the risk extends beyond accidental generation to intentional manipulation via prompt injection and jailbreaking. Adversaries can craft inputs that bypass safety aligners, forcing models to output malicious payloads or reveal proprietary system prompts, effectively weaponizing the assistant against the developer [28].

The propagation of these risks is exacerbated by the “illusion of correctness” instilled in developers, who often trust AI-generated code without rigorous verification. User studies demonstrate that developers utilizing AI assistants produce significantly less secure code and exhibit higher confidence in its safety compared to those working without assistance [6]. This trust deficit is compounded by the models’ inability to reliably reason about complex security semantics; even advanced models like GPT-4 struggle to distinguish between buggy and patched code versions in real-world scenarios, often providing non-deterministic and unfaithful reasoning [LLMs Cannot Reliably Identify and Reason About Security Vulnerabilities (Yet) A Comprehensive Evaluation, Framework, and Benchmarks]. Additionally, the training data itself remains susceptible to poisoning attacks, where adversaries inject malicious samples to teach models specific insecure behaviors, such as suggesting weak encryption algorithms, which persist even after defensive fine-tuning [16].

Recent benchmarks, such as FormAI and CyberSecEval, utilizing formal verification and automated testing, have confirmed that over 50% of AI-generated programs contain verifiable vulnerabilities, including memory safety errors and logic flaws [The FormAI Dataset Generative AI in Software Security Through the Lens of Formal Verification; Purple Llama CyberSecEval A Secure Cod-

ing Benchmark for Language Models]. While mitigation strategies like prompt engineering and reinforcement learning with semantic rewards show promise in hardening model outputs, they remain insufficient against adaptive adversaries who leverage LLMs to transform payloads and evade detection [Enhancing Security of AI-Based Code Synthesis with GitHub Copilot via Cheap and Efficient Prompt-Engineering; An LLM-Assisted Easy-to-Trigger Backdoor Attack on Code Completion Models: Injecting Disguised Vulnerabilities against Strong Detection]. Consequently, the software supply chain faces a paradigm shift where security Assurance must evolve from verifying static artifacts to auditing the probabilistic behaviors of generative engines and the integrity of their training ecologies. Future research must prioritize the development of robust, neuro-symbolic verification frameworks capable of certifying the security properties of AI-synthesized code prior to integration, thereby breaking the cycle of automated vulnerability propagation.

2.4 Evasion Attacks and Adversarial Robustness of AI Security Detectors

Bridging the gap between the generative risks of AI coding assistants and the systemic propagation of compromised dependencies, adversarial evasion attacks represent a critical threat vector targeting the integrity of AI-driven security detectors themselves. While the previous section highlighted how generative models inadvertently propagate vulnerabilities through stochastic code synthesis, this subsection examines how adversaries systematically manipulate input artifacts to induce misclassification in vulnerability detection systems, effectively rendering these defensive tools blind. Unlike traditional software obfuscation, these attacks leverage the differentiable nature of deep learning models to craft perturbations that exploit decision boundaries while preserving functional semantics. A primary vector involves semantic-preserving code transformations, where attackers alter syntax, control flow, or data structures without modifying program behavior to deceive graph neural network (GNN) and transformer-based classifiers. Research demonstrates that such techniques can significantly degrade detection performance; for instance, [29] introduces ALERT, a black-box attack generating adversarial examples that maintain naturalness to human judges while achieving high success rates against models like CodeBERT and GraphCodeBERT. Similarly, [30] utilizes refactoring tools to generate semantically equivalent variants, revealing that popular code embedding methods suffer performance drops of up to 9.58% under adversarial pressure.

The fragility of current detectors is further exacerbated by their reliance on superficial lexical features rather than deep semantic understanding, a limitation that complements the “illusion of correctness” observed in generative AI usage. [31] critically observes that many state-of-the-art models learn spurious correlations, such as specific variable names, rendering them highly susceptible to simple renaming attacks that constitute semantic-preserving transformations. This vulnerability extends to hardware security domains, where [32] employs reinforcement learning to generate adversarial circuit graphs that successfully evade GNN-based Trojan detection with 100% success rates. The arms race between attackers and defenders has also spurred the development of stealthier evasion techniques. While early adversarial examples were often easily detectable, modern approaches like [20] utilize adaptive triggers based on adversarial perturbations that bypass standard defense mechanisms with over 77% success rates, highlighting the insufficiency of current robustness measures.

Defensive strategies have largely centered on adversarial training and model hardening, yet these approaches face significant trade-offs between robustness and utility, mirroring the challenges of securing generative outputs against prompt injection. [33] established a foundational framework for evaluating classifier security, demonstrating that gradient-based attacks can systematically expose weaknesses in malware detection systems. In response, [29] shows that retraining models with

adversarial examples can improve accuracy against specific attacks by over 90%, though this often comes at the cost of increased false positives on benign inputs. Furthermore, [34] emphasizes that resource-constrained environments, typical in edge computing supply chains, cannot support computationally expensive defensive distillation or formal verification methods, leaving them uniquely vulnerable. The challenge is compounded by the lack of interpretability in robust models; [35] suggests that adversarial perturbations can be repurposed to explain model failures, yet integrating such explainability into real-time detection pipelines remains an open problem. Ultimately, the field must transition from static robustness evaluations to dynamic, adaptive defense mechanisms that account for the evolving sophistication of evasion tactics. This transition is essential not only for protecting individual detectors but also for preventing the cascading failures described in the following section, where compromised detection capabilities allow vulnerable components to permeate the broader AI software supply chain.

2.5 Supply Chain Propagation, Dependency Hijacking, and Systemic Risks

The systemic risk landscape in AI-enabled supply chains is defined by the cascading effects of compromised upstream components, where the interdependence of open-source libraries, pre-trained models, and automated generation tools creates a fragile ecosystem. Unlike traditional software dependencies, AI artifacts introduce unique propagation vectors where vulnerabilities in deep learning frameworks or foundational models can instantaneously affect thousands of downstream applications. Empirical studies on dependency networks reveal that transitive dependencies often obscure the true attack surface; for instance, in the Maven ecosystem, naive dependency analysis significantly overestimates affected projects compared to reachability analysis, yet fails to capture the nuanced propagation of logic flaws in AI kernels [11]. Similarly, in interpreted language ecosystems like npm and RubyGems, vulnerabilities propagate through complex dependency trees, with a significant proportion of projects exposed to indirect dependencies that remain unpatched despite available fixes [36].

The mechanism of propagation is further exacerbated by language-specific hijacking techniques and the dynamic nature of package resolution. Attackers exploit features such as Java’s dynamic class loading or npm’s install scripts to inject malicious code deep within the dependency graph, bypassing static build-time checks [37; 38]. The introduction of generative AI tools amplifies these risks through “package hallucinations,” where large language models (LLMs) confidently recommend non-existent or typosquatted packages, effectively automating the distribution of malicious dependencies at scale [9]. This phenomenon creates a feedback loop where AI-generated code integrates vulnerable or malicious libraries, which are then reused as training data for future models, perpetuating security anti-patterns across the supply chain [39].

Socio-technical factors play a critical role in facilitating rapid compromise. The reliance on trusted maintainer identities allows adversaries to leverage social engineering or account takeover to inject malicious updates into popular AI packages, a tactic exemplified by recent high-profile supply chain incidents [40]. Furthermore, the opacity of AI model reuse on platforms like Hugging Face complicates provenance tracking; pre-trained models often lack rigorous documentation regarding their training data or fine-tuning history, enabling the stealthy distribution of backdoored or trojanized artifacts [5; 26]. The integration of such models into production pipelines without integrity verification allows malware embedded in neural weights to execute upon deserialization, bypassing traditional signature-based defenses.

Addressing these systemic risks requires a shift from component-level scanning to holistic ecosys-

tem monitoring. Current Software Composition Analysis (SCA) tools often suffer from high false positive rates and fail to account for the specific semantics of AI dependencies [41]. Future defense mechanisms must integrate dynamic reachability analysis with behavioral monitoring of AI artifacts, leveraging techniques such as graph-based learning to detect anomalous dependency patterns and metadata analysis to identify malicious packages before they permeate the ecosystem [42; 43]. Ultimately, securing the AI supply chain demands a unified framework that bridges the gap between software engineering best practices and machine learning safety, ensuring transparency and accountability from data lineage to model deployment.

3 Artificial Intelligence-Driven Methodologies for Automated Vulnerability Detection

3.1 Semantic Code Representation and Graph-Based Learning Architectures

The transition from rule-based static analysis to data-driven vulnerability detection necessitates robust semantic code representations that capture the intrinsic logic of software beyond superficial syntax. Foundational to this paradigm shift is the encoding of source code and binary artifacts into structural graphs, such as Abstract Syntax Trees (ASTs) and Control Flow Graphs (CFGs), which preserve critical data and control dependencies. Early methodologies like [44] demonstrated that graph-based deep learning could reason over program structures to identify variable misuse, outperforming sequence-only models by capturing long-range dependencies. Building on this, [45] introduced a general Graph Neural Network (GNN) framework that learns comprehensive program semantics from rich code representations, achieving significant accuracy improvements over state-of-the-art baselines by effectively modeling inter-procedural vulnerabilities. Similarly, [3] established a systematic framework for obtaining syntax and semantics-based vector representations, proving essential for detecting complex vulnerabilities in C/C++ programs that traditional methods miss.

A persistent challenge in semantic representation is the mitigation of variable name bias, where models memorize specific identifiers rather than learning functional patterns. To address this, [46] treats source code as a flat sequence of tokens initialized by pre-trained embeddings, leveraging residual connections to build robust graph representations independent of specific naming conventions. Further advancing this, [47] integrates a “Poacher Flow” to bridge dynamic and static analysis gaps, reducing classifier bias through focal loss optimization. The granularity of detection has also evolved from function-level classification to precise statement-level localization. [48] formulates vulnerability detection as a node classification task on program dependency graphs, significantly improving F1-scores by resolving conflicts between function-level status and statement-level contributions. Complementing this, [49] utilizes intermediate code and granularity refinement to accommodate semantic relations across files, enabling high-precision localization of vulnerability origins.

In the binary domain, where source code is unavailable, semantic representation relies on parsing assembly instructions into executable graphs. [50] pioneered the use of neural networks to generate embeddings from CFGs for cross-platform binary similarity detection, drastically reducing computation time while enhancing accuracy compared to approximate graph matching. [51] further integrates semantic learning with emulation, using labeled semantic flow graphs to filter candidate functions before dynamic signature comparison, achieving superior search accuracy for known CVEs. For heterogeneous environments, [52] proposes multi-level heterogeneous graph embeddings that capture complex meta-paths in smart contracts, facilitating fine-grained line-level detection of logic flaws. However, standard GNNs often struggle with long-distance dependencies in deep code

graphs. [53] addresses this via graph simplification and kernel-scaled representation modules to capture global information, while [54] employs meta-path based attention to model heterogeneous relations effectively. Despite these advances, [55] cautions that many models still rely on spurious features, highlighting the need for representations that enforce true semantic understanding rather than superficial pattern matching. Future architectures must therefore synergize structural fidelity with semantic depth to ensure robust generalization across diverse software ecosystems.

3.2 Large Language Models and Prompt Engineering for Zero-Shot Detection

The transition from rigid structural encodings to flexible semantic understanding culminates in the integration of Large Language Models (LLMs) into vulnerability detection, representing a paradigmatic shift from specialized, fine-tuned classifiers to generalist agents capable of zero-shot and few-shot reasoning. Unlike the traditional deep learning approaches discussed previously, which rely heavily on task-specific feature engineering or explicit graph-based structural encodings, LLMs leverage massive pre-training on diverse code corpora to internalize semantic patterns associated with security flaws [56]. This capability allows for the identification of Common Weakness Enumeration (CWE) types across multiple programming languages without the need for extensive labeled datasets, addressing the chronic scarcity of high-quality vulnerability data [57].

Central to leveraging this generalist capability is the art of prompt engineering, which serves as the primary interface for guiding model behavior where explicit structural graphs are absent. Early evaluations indicated that naive prompting strategies often yield performance comparable to random guessing, particularly on complex, real-world datasets [58; 59]. However, advanced prompting techniques have significantly narrowed this gap. Strategies such as role-based prompting, where the model is instructed to act as a security auditor, and the inclusion of structural auxiliary information have demonstrated efficacy in enhancing detection accuracy [60; 61]. Furthermore, the incorporation of Chain-of-Thought (CoT) reasoning enables models to decompose complex vulnerability analysis into sequential logical steps, tracing data flows and logical conditions that single-pass classifiers frequently miss [62; 63].

Comparative analyses reveal a nuanced landscape where LLMs excel in recall but often struggle with precision, generating higher false positive rates compared to traditional Static Application Security Testing (SAST) tools [64]. While SAST tools provide rigorous guarantees based on formal rules, they lack the semantic understanding to detect logic errors or inter-procedural vulnerabilities that LLMs can infer through context [65]. To mitigate the limitations of pure zero-shot approaches and address these precision deficits, recent frameworks propose hybrid architectures. For instance, Retrieval-Augmented Generation (RAG) systems like Vul-RAG enhance LLM reasoning by retrieving relevant vulnerability knowledge from curated databases, thereby grounding the model’s output in verified security patterns [66]. Similarly, frameworks such as DLAP combine the semantic depth of LLMs with the structural precision of deep learning models to achieve superior detection performance [67].

Despite these advancements, significant challenges remain regarding robustness and generalization, highlighting the limitations of purely probabilistic approaches. Studies indicate that LLMs are susceptible to adversarial perturbations, such as variable renaming or the insertion of irrelevant code, which can degrade detection accuracy by over 12% [68]. Moreover, the performance of LLMs varies significantly across different CWE categories, often failing to distinguish between buggy and fixed versions of code in nuanced scenarios [69]. These reliability concerns and the lack of formal guarantees in current LLM-based methods underscore the necessity for integrating

rigorous verification mechanisms. Consequently, future research directions must focus on developing more robust prompting strategies and, crucially, integrating formal verification to validate LLM-generated hypotheses, creating standardized benchmarks that accurately reflect the complexity of modern software supply chains [70; 71]. This evolution towards verified, hybrid systems sets the stage for the neuro-symbolic frameworks discussed next.

3.3 Neuro-Symbolic Hybrids and Formal Verification Integration

Neuro-symbolic hybrid frameworks represent a pivotal evolution in automated vulnerability detection, synthesizing the scalable pattern recognition capabilities of deep learning with the mathematical rigor of symbolic execution and formal verification. While purely data-driven approaches often suffer from high false positive rates and a lack of interpretability, integrating symbolic reasoning provides a mechanism to generate concrete counterexamples and verify safety properties with high assurance. A primary application of this synergy is neural-guided symbolic execution, which addresses the path explosion problem inherent in traditional model checking. By leveraging neural networks to predict likely vulnerable paths or critical constraints, these hybrids can direct symbolic explorers toward deep-state bugs more efficiently. For instance, [72] demonstrates this by utilizing machine learning to rank program paths based on criticalness, applying symbolic execution only to top-ranked candidates to achieve scalability without sacrificing precision. Similarly, [73] employs a two-stage process where program slicing identifies suspicious dependencies, which are subsequently verified via symbolic execution to eliminate spurious warnings.

Beyond guiding exploration, neuro-symbolic integration is essential for the formal verification of AI-generated code and the models themselves. As generative tools increasingly populate the software supply chain, ensuring the correctness of their output is paramount. [70] proposes a closed-loop framework where Bounded Model Checking (BMC) generates counterexamples for vulnerabilities detected in C programs, which are then fed back to a Large Language Model (LLM) to synthesize repairs that are formally verified before deployment. This approach is further substantiated by [74], which utilizes SMT-based model checking to definitively label vulnerabilities in AI-generated C code, creating a ground-truth dataset that eliminates the ambiguity of probabilistic classification. Furthermore, the verification scope extends to the neural networks themselves; [34] highlights the growing necessity of applying formal methods to verify safety properties of trained models against adversarial perturbations and quantization errors, ensuring reliability in safety-critical deployments.

Advanced hybrid architectures also explore mixing symbolic and neural representations within constraint solvers to handle non-linear relationships that pure symbolic engines struggle with. [75] exemplifies this by using AI agents to transform informal specifications into formal languages, subsequently employing symbolic methods to synthesize test inputs that validate the generated parsers, thereby ensuring provable correctness. In the domain of explainability, neuro-symbolic techniques move beyond saliency maps to provide causal evidence. [76] introduces counterfactual reasoning to identify minimal perturbations in code graphs that alter model predictions, offering developers actionable insights into root causes rather than opaque probability scores. Additionally, [77] critiques existing explanation methods for poor accuracy in locating vulnerable lines, suggesting that integrating program slicing—a symbolic technique—with deep learning is necessary to achieve precise localization. Despite these advances, challenges remain in balancing the computational overhead of symbolic solvers with the speed of neural inference. Future research must focus on optimizing these interfaces, particularly for resource-constrained environments, and developing unified ontologies that allow seamless information exchange between neural heuristics and symbolic logic engines.

3.4 Specialized Detection Paradigms for Emerging Domains and Artifacts

Building on the neuro-symbolic foundations and formal verification strategies outlined previously, the expanding attack surface of modern software supply chains necessitates specialized detection paradigms that transcend traditional source code analysis. These approaches must target non-traditional artifacts, including blockchain smart contracts, deep learning frameworks, and AI-generated code, where standard static analyzers often fall short. In the domain of blockchain, the immutability of deployed contracts demands high-precision detection of logic flaws such as reentrancy and timestamp dependence. While early approaches relied on rigid expert rules, recent methodologies integrate Graph Neural Networks (GNNs) with domain-specific knowledge to capture complex control-flow semantics. For instance, frameworks combining GNNs with expert patterns have demonstrated significant accuracy improvements by normalizing contract graphs and utilizing temporal message propagation to identify critical nodes [78]. Similarly, Eth2Vec leverages neural language processing on Ethereum Virtual Machine (EVM) bytecodes to achieve robustness against code rewrites, outperforming feature-engineered static tools [79]. More recently, Large Language Models (LLMs) have been adapted for this niche; GPTScan combines GPT’s semantic understanding with static confirmation to detect logic vulnerabilities missed by pattern-based tools, achieving high precision in token contract audits [80]. Furthermore, adversarial frameworks like GPTLens employ a dual “auditor-critic” mechanism to balance the trade-off between recall and false positives in LLM-driven auditing [81].

Beyond blockchain, securing the AI supply chain requires detecting vulnerabilities within deep learning frameworks and their underlying kernels, a challenge that aligns with the need for robust model verification discussed earlier. Traditional fuzzing often struggles with the complex input structures of ML libraries; however, neural-guided fuzzing techniques, such as those utilizing LSTMs to predict optimal mutation locations based on code coverage, have shown efficacy in uncovering memory safety errors in diverse input formats [82]. This is critical for securing the C/C++ kernels of popular ML frameworks, where type-aware mutation and neural guidance can expose API misuse that static analyzers overlook. Concurrently, the proliferation of AI-generated code introduces a new vector for vulnerability propagation, reinforcing the reliability concerns addressed in prior sections. Empirical studies indicate that generative models, trained on vast corpora of open-source code, frequently reproduce insecure patterns; for example, assessments of GitHub Copilot reveal that approximately 40% of its suggestions contain vulnerabilities spanning various CWE categories [27]. To mitigate this, specialized benchmarks like CyberSecEval and FormAI have been developed to rigorously evaluate the security posture of LLMs, utilizing formal verification to label AI-generated C programs with definitive vulnerability ground truths [83; 74].

Emerging detection strategies for these diverse artifacts increasingly rely on the neuro-symbolic hybrids and retrieval-augmented generation (RAG) architectures introduced in the preceding section. Systems like Vul-RAG enhance LLM reasoning by retrieving relevant vulnerability knowledge from CVE instances, substantially improving detection accuracy over baseline models [66]. Additionally, the scope of detection has extended to hardware description languages (HDLs), where LLMs are evaluated on their ability to identify CWEs in SystemVerilog RTL code, revealing that nearly 60% of AI-generated hardware designs are prone to security flaws [84]. Despite these advances, challenges remain in generalizing across diverse domains and reducing false positives without sacrificing coverage. Future research must focus on developing unified ontologies that bridge the semantic gaps between smart contracts, ML kernels, and hardware designs, while integrating formal verification guarantees directly into generative pipelines to ensure security-by-design in emerging software ecosystems.

4 Securing the Artificial Intelligence Model Supply Chain and Development Pipelines

4.1 Provenance, Lineage, and Accountability Artifacts for AI Models

The integrity of the artificial intelligence supply chain hinges on the ability to rigorously track the lineage of model artifacts from raw data ingestion to final deployment. Unlike traditional software, where dependencies are discrete and static, AI models represent a complex fusion of data, hyperparameters, and architectures, necessitating an evolved provenance framework that extends beyond the conventional Software Bill of Materials (SBOM). The emergence of the AI Bill of Materials (AIBOM) addresses this gap by capturing the nuanced interdependencies between training corpora, preprocessing pipelines, and model weights, thereby enabling granular traceability essential for regulatory compliance and vulnerability response [85]. Central to this paradigm is the Data Bill of Materials (DataBOM), which immutably records the origin and transformation history of training datasets. Given that AI code generators and foundation models are increasingly vulnerable to targeted data poisoning attacks where malicious samples are injected into unsanitized sources, establishing a cryptographically verifiable chain of custody for data is critical to prevent the propagation of compromised behaviors [39; 18].

To operationalize accountability, the community has converged on standardized documentation artifacts such as Model Cards and FactSheets, which declare model purpose, performance metrics, and safety evaluations. However, static documentation is insufficient for dynamic supply chains; recent proposals advocate for continuous AI auditing infrastructures, such as AuditMAI, which integrate provenance tracking directly into MLOps pipelines to facilitate real-time assurance rather than one-off assessments [86]. This shift is particularly vital given the “dislocated accountabilities” observed in modern AI development, where modular supply chains often obscure responsibility among diverse actors, leaving gaps that ethical guidelines alone cannot bridge [87; 88]. By anchoring accountability in technical artifacts, organizations can better navigate the complex liability landscapes defined by emerging regulations like the EU AI Act, which mandates strict transparency for high-risk systems [89].

Beyond regulatory adherence, provenance mechanisms serve as a fundamental defense against intellectual property theft and unauthorized model derivation. Techniques such as “Model DNA” and robust watermarking allow developers to establish causal links between foundation models and downstream derivatives, detecting unauthorized fine-tuning or model stealing even when weights are slightly perturbed [90]. Furthermore, in the context of open-source model distribution, where pre-trained weights may conceal self-extracting malware or backdoors, provenance verification acts as a primary trust anchor before model loading [26; 91]. Yet, significant challenges remain regarding the granularity and accessibility of provenance data. While black-box audits offer limited visibility, true accountability requires “outside-the-box” access to training methodologies and internal evaluations, suggesting that future provenance standards must enforce deeper transparency without compromising proprietary secrets [92]. Ultimately, the convergence of blockchain-enabled SBOMs, continuous auditing, and cryptographic watermarking forms the bedrock of a trustworthy AI ecosystem, transforming provenance from a passive record into an active mechanism for security and governance [93].

4.2 Cryptographic Attestation and Integrity Verification Mechanisms

Building on the foundational traceability established by AIBOMs and continuous auditing, ensuring the integrity of artificial intelligence models within complex supply chains necessitates a

paradigm shift from traditional software signing to sophisticated cryptographic attestation mechanisms capable of verifying both the artifact and the process of its creation. As the reliance on third-party pre-trained models grows, the threat landscape has expanded to include weight manipulation, steganographic malware embedding, and the distribution of trojanized artifacts [26]. Consequently, robust defense strategies must mathematically guarantee that deployed models remain unaltered and originate from trusted training environments, transitioning from passive documentation to active verification.

A primary avenue for achieving trustless verification involves Zero-Knowledge Proofs (ZKPs), which allow model providers to attest to specific properties of their training data and procedures without exposing proprietary weights or sensitive input corpora. This approach addresses the critical tension between transparency and intellectual property protection, enabling auditors to verify the absence of copyrighted material or the diversity of training sets while maintaining confidentiality [92]. Recent frameworks propose utilizing ZKPs to generate verifiable evidence that a model was trained according to a specified algorithm and dataset, effectively creating a cryptographic receipt of the training lifecycle. This is particularly vital given the susceptibility of outsourced training to backdoor injection, where adversaries manipulate models to behave maliciously on specific triggers while maintaining high accuracy on benign inputs [94]. By cryptographically binding the training process to the final artifact, ZKPs mitigate the risk of “handcrafted” backdoors inserted via direct weight manipulation, a technique shown to evade many standard detection defenses [24].

Complementing cryptographic proofs, hardware-assisted attestation via Trusted Execution Environments (TEEs) provides a root of trust for the computational infrastructure itself. TEEs isolate the training and inference processes from the host operating system and hypervisor, generating remote attestation reports that certify the code integrity and the isolation of the execution environment. This hardware-based approach is essential for defending against sophisticated supply chain attacks where adversaries compromise the underlying infrastructure to inject malicious payloads or extract model parameters [34]. However, the integration of TEEs introduces performance overheads and requires careful management of key provisioning, presenting trade-offs between security assurance and operational efficiency. Furthermore, while TEEs protect the runtime environment, they must be coupled with secure model serialization formats to prevent arbitrary code execution during deserialization, a known vulnerability in common formats like Pickle [26].

To detect post-deployment tampering, digital watermarking and fragile embedding techniques offer a mechanism for continuous integrity monitoring. Unlike robust watermarks designed for ownership assertion, fragile watermarks are engineered to break upon any modification of the model weights, thereby localizing bit-flip attacks or unauthorized parameter updates. Recent advancements in resilient watermarking for code-generating models demonstrate the feasibility of embedding semantic-preserving transformations that survive refactoring yet remain detectable [90]. Extending this concept to neural weights, adaptive watermarking schemes can serve as canaries in the coal mine, signaling integrity compromises before malicious behavior manifests. Nevertheless, these methods face challenges against adaptive adversaries who may attempt to remove watermarks through model pruning or fine-tuning, necessitating a co-design of watermarking schemes with robust training protocols.

Looking forward, the convergence of these technologies points toward a holistic attestation framework that combines ZKPs for process verification, TEEs for environmental assurance, and fragile watermarking for artifact integrity. Such a multi-layered approach is critical for establishing a verifiable chain of custody in AI supply chains, moving beyond static signatures to dynamic, mathematically grounded guarantees of trustworthiness. While these mechanisms provide the cryptographic

bedrock for trust, their operational realization requires embedding these guarantees directly into the workflows that build and deploy models. Future research must address the scalability of ZKPs for large-scale models and the standardization of attestation protocols across diverse hardware architectures to realize a truly secure AI ecosystem, thereby laying the necessary groundwork for the hardening of MLOps pipelines discussed in the following section.

4.3 Hardening AI-Specific Continuous Integration and Deployment Pipelines

The hardening of AI-specific Continuous Integration and Continuous Deployment (CI/CD) pipelines, often termed MLOps, necessitates a fundamental reimagining of traditional DevSecOps paradigms to address the unique non-deterministic and data-centric nature of machine learning workflows. Unlike conventional software pipelines where build artifacts are deterministic functions of source code, ML pipelines orchestrate complex interactions between data ingestion, hyperparameter tuning, model training, and deployment, creating an expanded attack surface vulnerable to supply chain compromises. A primary concern is the integrity of the training infrastructure itself; adversaries may target compromised training nodes or exploit unauthorized compute usage to manipulate model weights during the learning phase. To mitigate this, recent proposals advocate for firmware-based offline licensing for AI chips and rigorous integrity verification protocols for distributed training jobs across untrusted cloud environments, ensuring that the computational substrate remains trustworthy [34]. Furthermore, the pipeline’s dependency graph introduces critical risks, as malicious components injected into container images or library dependencies can poison the entire training process before a single epoch is completed.

Defending against pipeline poisoning requires a shift from static binary analysis to dynamic behavioral monitoring of the training lifecycle. Attacks such as “TrojanPuzzle” demonstrate how adversaries can bypass static dataset cleansing by planting malicious triggers in out-of-context regions like docstrings, which are then learned by the model during training [16]. Consequently, securing the pipeline demands real-time validation of data provenance and model lineage. The concept of a Data Bill of Materials (DataBOM) has emerged as a critical control, enabling the tracking of dependency relationships between raw datasets, preprocessing steps, and final model artifacts to prevent the use of compromised training corpora [95]. Empirical studies on the evolution of CI/CD configurations in ML projects reveal a prevalence of misconfigurations and a lack of standardized testing frameworks, which exposes pipelines to build-time attacks and reproducibility failures [96]. Addressing these gaps requires the integration of automated security gates that verify not only code integrity but also statistical properties of the training data and the resulting model weights.

Moreover, the deployment stage presents distinct challenges where models, once certified, may be substituted or tampered with before reaching production. The threat of deployment-stage backdoor attacks, where adversaries selectively modify model weights to embed triggers in user devices, underscores the need for cryptographic attestation and runtime verification mechanisms within the deployment pipeline [25]. Innovative approaches leverage Trusted Execution Environments (TEEs) to generate verifiable evidence that model training and inference occurred in isolated, unmodified environments, thereby protecting against hypervisor-level attacks [93]. Additionally, the integration of AI-driven security tools themselves into the pipeline must be scrutinized; as large language models are increasingly used to generate pipeline scripts or patch vulnerabilities, they introduce risks of propagating insecure patterns if the models themselves have been poisoned [39]. Future research must focus on developing adaptive defense-in-depth strategies that combine formal verification of pipeline configurations with continuous monitoring of model drift and anomaly detection, ensuring resilience against both external supply chain attacks and internal configuration errors [97].

Ultimately, achieving a secure MLOps ecosystem requires a holistic synthesis of cryptographic guarantees, behavioral analysis, and rigorous governance standards tailored to the probabilistic nature of AI systems.

4.4 Detection and Mitigation of Malicious Assets in Model Ecosystems

Building upon the procedural groundwork and cryptographic guarantees established in secure MLOps pipelines, the operational realization of these defenses relies heavily on automated methodologies capable of detecting malicious assets within the expansive ecosystem of public model registries and package managers. While the previous section addressed the hardening of the build and deployment lifecycle, this subsection surveys the landscape of detecting trojanized libraries, compromised models, and malicious packages, emphasizing the critical transition from traditional static analysis to behavioral profiling and Large Language Model (LLM)-driven intelligence. The primary challenge in this domain lies in distinguishing between benign complexity and adversarial obfuscation within high-dimensional artifacts, a necessity that precedes the collaborative privacy paradigms discussed subsequently.

Static analysis and behavioral monitoring remain foundational for identifying malicious patterns in interpreted language ecosystems. Tools like [98] leverage behavior sequence knowledge mapping to detect malicious npm packages by reconstructing code and analyzing dynamic API call sequences, effectively identifying obfuscated content that static signatures miss. Similarly, [38] highlights metadata anomalies, such as expired maintainer domains and suspicious install scripts, as critical indicators of supply chain compromise. These approaches are complemented by empirical studies in the PyPI ecosystem, where [4] reveals that over 74% of malicious packages infiltrate projects via source code installation, often exhibiting multi-stage behaviors like credential harvesting and command execution. To counter such dynamic threats, [42] introduces a deep graph learning approach that constructs data provenance graphs from system calls during installation, achieving 96% accuracy in flagging malicious installers by analyzing the temporal and causal relationships of processes. Furthermore, [37] addresses runtime integrity in Java environments by enforcing allowlists derived from Software Bill of Materials, effectively mitigating dynamic class loading attacks that bypass build-time checks.

Beyond traditional software packages, the integrity of model artifacts themselves is increasingly targeted, representing a unique vector in the AI supply chain. [26] demonstrates the severity of this threat through MaleficNet 2.0, a technique that embeds self-extracting malware directly into neural network weights using spread-spectrum coding, rendering the model a vector for arbitrary code execution upon loading. Detecting such stealthy injections requires moving beyond file hashing to semantic and structural analysis. Here, LLMs offer transformative potential for threat hunting, extending the verification capabilities introduced in pipeline security. [99] illustrates how property-specific guidance can be used to audit code generation models, while [100] provides benchmarks for evaluating the offensive and defensive capabilities of LLMs in identifying security risks. LLMs can also automate the parsing of security advisories and the clustering of malicious patterns; for instance, [101] utilizes NLP to construct security knowledge graphs, extracting high-fidelity threat behaviors from unstructured reports to identify coordinated campaigns.

However, significant trade-offs exist between detection precision and scalability, mirroring the efficiency challenges found in privacy-preserving computation. While static tools offer low false-positive rates, they struggle with novel obfuscation techniques, as noted in [41], which found substantial variance in vulnerability reporting across industry tools. Conversely, LLM-driven approaches, while

powerful in zero-shot classification and reasoning [68], face challenges regarding hallucination and computational overhead. Moreover, adversarial evasion remains a critical concern; [33] and [102] warn that detectors themselves are vulnerable to gradient-based evasion, necessitating robust training paradigms. Future research must therefore focus on hybrid architectures that combine the deterministic guarantees of provenance tracking [103] with the semantic reasoning of LLMs. Creating such resilient defense mechanisms is essential not only for securing standalone assets but also for safeguarding the shared training data and model updates central to the collaborative frameworks explored in the following section.

4.5 Privacy-Preserving Collaborative Development and Training Frameworks

The integration of artificial intelligence into critical supply chains necessitates collaborative development paradigms that reconcile the utility of shared model training with the imperative of data sovereignty. Privacy-preserving frameworks address this by decentralizing the training process, thereby mitigating the attack surface associated with centralized data repositories where proprietary information is vulnerable to exfiltration or poisoning. Federated Learning (FL) has emerged as a primary architectural pattern, enabling stakeholders to compute model updates locally while aggregating only gradients or weights globally. However, the security of FL is not intrinsic; recent analyses indicate that FL setups remain susceptible to gradient inversion and membership inference attacks, where adversaries can reconstruct sensitive training data from shared updates [104]. Consequently, robust FL implementations must incorporate defensive strategies such as secure aggregation protocols and differential privacy mechanisms to sanitize gradients before transmission, ensuring that individual contributions cannot be reverse-engineered by malicious participants or compromised servers [104].

Beyond standard FL, Secure Multi-Party Computation (MPC) offers a rigorous cryptographic alternative for distributed training over vertically or horizontally partitioned data. MPC protocols allow multiple parties to jointly compute functions over their inputs without revealing those inputs, providing formal privacy guarantees that are mathematically verifiable. While MPC eliminates the need to trust a central aggregator, it introduces significant computational overhead and communication latency, presenting a critical trade-off between security strength and operational efficiency in resource-constrained environments [34]. This tension is particularly acute in edge computing scenarios, where the constraints on memory and power limit the feasibility of complex cryptographic operations, necessitating lightweight protocols that maintain security guarantees without exhausting device capabilities [34]. Furthermore, collaborative frameworks must defend against insider threats, such as model poisoning by malicious clients who inject biased or backdoored updates to compromise the global model’s integrity [13].

The scope of privacy preservation extends beyond training to include inference and model ownership verification. In collaborative ecosystems, protecting the intellectual property of the model itself is as crucial as protecting the input data. Techniques such as digital watermarking and fragile embeddings are increasingly integrated into privacy-preserving pipelines to assert ownership and detect unauthorized parameter modifications or extraction attempts [26]. Additionally, privacy-preserving inference mechanisms, utilizing homomorphic encryption or Trusted Execution Environments (TEEs), enable users to query remote models without exposing sensitive input data to the model owner, a capability essential for secure AI-as-a-Service deployments in regulated industries [34]. Despite these advancements, current frameworks often struggle to balance the triad of privacy, utility, and robustness. For instance, aggressive differential privacy noise can degrade model accuracy, while strict MPC constraints can render training impractically slow. Future research

must focus on adaptive hybrid architectures that dynamically adjust cryptographic rigor based on threat models and data sensitivity, alongside the development of standardized benchmarks to evaluate the resilience of these collaborative frameworks against sophisticated adaptive adversaries [93]. Ultimately, achieving trustworthy collaborative AI requires a holistic approach that embeds privacy and security primitives directly into the learning algorithms rather than treating them as peripheral add-ons [95].

5 Automated Remediation, Risk Prioritization, and Operational Defense Strategies

5.1 Generative AI for Automated Vulnerability Remediation and Patch Synthesis

The paradigm of automated vulnerability remediation has shifted from rigid, rule-based program repair to dynamic, generative synthesis driven by Large Language Models (LLMs). This transition leverages sequence-to-sequence architectures to translate vulnerable code snippets into secure equivalents, treating repair as a neural machine translation task [105]. Early approaches utilizing pre-trained models like CodeT5 and PLBART demonstrated the feasibility of learning fix patterns from massive repositories, yet often struggled with the semantic complexity of security flaws compared to functional bugs [106]. Recent advancements have addressed these limitations through sophisticated fine-tuning strategies; for instance, parameter-efficient techniques such as LoRA and IA3, when applied to datasets of confirmed vulnerability fixes, have been shown to enhance secure code generation by approximately 6% in C and C++ contexts [107].

A critical evolution in this domain is the integration of structural awareness and retrieval mechanisms to overcome the “black-box” nature of pure generative models. Frameworks like NAVRepair explicitly incorporate Abstract Syntax Tree (AST) node-type information and Common Weakness Enumeration (CWE) templates to guide the repair process, achieving a 26% accuracy improvement over standard LLM baselines for C/C++ vulnerabilities [108]. Similarly, retrieval-augmented generation (RAG) approaches, such as RAP-Gen, mitigate the constraints of fixed parametric knowledge by retrieving relevant historical fix patterns to augment the generation context, significantly outperforming state-of-the-art methods on benchmarks like Defects4J [109]. Furthermore, multi-LLM collaboration frameworks like VulMaster synthesize expert knowledge from CWE systems with structural code representations, doubling the effectiveness of vulnerability repair metrics compared to single-model approaches [110].

To ensure functional correctness and security validity, modern systems increasingly employ feedback-driven iterative refinement loops. By coupling LLMs with static analysis tools or formal verifiers, these systems create a self-correcting mechanism where compiler errors and security warnings guide subsequent generation steps. The ESBMC-AI framework exemplifies this neuro-symbolic synergy, using bounded model checking to generate counterexamples that inform the LLM’s repair prompts, resulting in an 80% success rate for fixing buffer overflows and pointer dereferences [70]. Additionally, reinforcement learning paradigms have been introduced to optimize patch quality; SecRepair utilizes semantic reward mechanisms to prioritize security-hardening modifications, ensuring that generated patches address root causes rather than merely suppressing symptoms [111].

Despite these advances, challenges persist regarding the reliability of zero-shot repairs and the propagation of training data biases. Studies indicate that while LLMs can collectively repair synthetic scenarios, their performance on real-world, complex vulnerabilities remains inconsistent, often failing to preserve functionality or introducing new defects [112]. Moreover, the risk of “un-repairing”

code via data poisoning attacks highlights the fragility of models trained on uncured open-source data, necessitating rigorous dataset sanitation [18]. Future research must therefore focus on developing robust, verifiable generation pipelines that combine the semantic flexibility of LLMs with the mathematical guarantees of formal methods, moving toward truly autonomous, self-healing software supply chains.

5.2 Dynamic Risk Prioritization and Exploitability Prediction Models

Dynamic risk prioritization and exploitability prediction represent a critical paradigm shift from static severity scoring to probabilistic, context-aware threat assessment, forming the analytical foundation for the automated remediation strategies discussed subsequently. Traditional metrics like the Common Vulnerability Scoring System (CVSS) often fail to correlate with actual exploitation rates due to their reliance on static characteristics rather than empirical threat intelligence [113]. To address this, modern methodologies leverage machine learning to augment scoring systems with dynamic features derived from historical exploit data, social media sentiment, and dependency graph topology. A seminal advancement in this domain is the Exploit Prediction Scoring System (EPSS), which utilizes crowd-sourced expertise and data-driven models to achieve an 82% performance improvement over prior methods in distinguishing vulnerabilities exploited in the wild [114]. By integrating external threat feeds and natural language processing (NLP) of vulnerability descriptions, these models forecast the probability of weaponization, enabling organizations to triage remediation efforts based on likelihood rather than hypothetical impact [115].

Beyond textual analysis, the structural context of vulnerabilities within complex dependency trees significantly influences risk profiles, providing the granular insight necessary for the targeted repair mechanisms outlined in the next section. Research indicates that naive dependency-based analysis often inflates risk by ignoring reachability; however, incorporating fine-grained call graph analysis and transitive dependency resolution allows for more precise impact assessment [11]. Graph Neural Networks (GNNs) and attention mechanisms have been employed to model these dependency relationships, capturing how a vulnerability’s position within the supply chain amplifies or mitigates its threat potential [10]. Furthermore, linking vulnerabilities to adversary tactics via frameworks like MITRE ATT&CK provides a threat-centric ranking mechanism that aligns remediation with organizational defense strategies, yielding significant improvements in identifying high-priority targets [116; 117].

Despite these advances, predictive models face substantial challenges related to concept drift and data quality, which directly impact the reliability of downstream automated fixes. The temporal evolution of vulnerability characteristics necessitates adaptive models capable of handling out-of-vocabulary terms and shifting attacker behaviors without frequent retraining [118]. Additionally, the inherent noise in historical datasets—where vulnerabilities are often discovered long after introduction—can mislead classifiers, prompting techniques that focus learning on known vulnerable components and their fixes to reduce label ambiguity [119]. Economic factors also play a critical role; empirical studies of cybercrime markets reveal strong correlations between market activity, exploit pricing, and the likelihood of attack realization, suggesting that economic indicators should be integrated into risk models [120].

Future directions must address the scarcity of ground-truth exploitation data and the need for explainable prioritization to ensure that the high-probability threats identified are actionable for both human analysts and autonomous agents. While models like [121] offer interpretability for severity scoring, similar transparency is required for exploitability predictions to foster trust among

security practitioners. Moreover, as generative AI accelerates both vulnerability discovery and patch deployment, the dynamics of the “patch vs. exploit” race require updated mathematical models to inform policy and resource allocation [122]. Ultimately, effective dynamic prioritization demands a holistic synthesis of semantic analysis, graph topology, economic signals, and temporal dynamics, serving as the essential precursor to the generative repair and runtime defense capabilities that follow.

5.3 Runtime Defense, Anomaly Detection, and Self-Healing Architectures

Runtime defense strategies represent the final line of protection in the AI software supply chain, shifting focus from static pre-deployment analysis to dynamic, real-time monitoring and autonomous response. As traditional signature-based methods fail to detect novel zero-day exploits and adaptive adversarial attacks, unsupervised learning techniques have emerged as critical mechanisms for identifying anomalies in system behavior and data distributions. Deep learning-based anomaly detection systems, particularly those utilizing autoencoders and recurrent neural networks, model normal operational baselines to flag deviations indicative of intrusion or model poisoning. For instance, frameworks like DeepAID provide interpretable explanations for unsupervised detections, bridging the gap between black-box alerts and actionable security insights [123]. Similarly, manifold-based approaches leverage variational autoencoders to learn low-dimensional structures of benign data, enabling robust runtime monitoring that assesses the trustability of system outputs without requiring labeled attack data [124]. However, the efficacy of these methods is often contingent on the quality of training data; studies warn that models trained on known attacks may yield a false sense of security when encountering unseen threat vectors, necessitating rigorous evaluation against unknown attack scenarios [125].

Beyond detection, the paradigm of operational defense is evolving towards autonomous incident response through Deep Reinforcement Learning (DRL). DRL agents are increasingly deployed to navigate complex state spaces, learning optimal mitigation policies that balance security enforcement with operational continuity. In industrial control systems and cyber-physical environments, RL agents have demonstrated the ability to generate minimal test patterns for hardware trojan detection and execute dynamic response strategies that isolate compromised nodes while maintaining service availability [126; 127]. Furthermore, DRL has been formalized to automate penetration testing and fuzzing, where agents learn to exploit vulnerabilities to proactively identify and patch weaknesses before malicious actors can leverage them [128; 129]. These autonomous capabilities are essential for scaling defense mechanisms in high-velocity environments where human reaction times are insufficient.

The culmination of these technologies is the emergence of self-healing architectures capable of restoring system integrity without human intervention. By integrating large language models (LLMs) with formal verification tools, modern systems can not only detect runtime errors but also synthesize and validate patches in real-time. Approaches combining bounded model checking with LLM-driven code generation have achieved significant success rates in automatically repairing buffer overflows and pointer dereferences, effectively closing the loop between detection and remediation [70]. Additionally, bio-inspired and virtual structure-based mechanisms allow critical infrastructures to dynamically reconfigure network topologies or restart services upon detecting faults, ensuring resilience against persistent threats [130]. Despite these advances, challenges remain in ensuring the reliability of autonomous agents, particularly regarding the risk of AI-induced errors during self-repair processes and the need for robust safeguards against prompt injection in AI-driven response systems [131]. Future research must therefore prioritize the development of ver-

ifiable self-healing protocols that maintain strict safety guarantees while adapting to the evolving threat landscape of the AI supply chain.

5.4 Human-AI Collaboration, Trust Calibration, and Operational Governance

Building upon the self-healing architectures and autonomous response mechanisms discussed previously, the integration of artificial intelligence into operational defense necessitates a critical paradigm shift from fully autonomous systems to synergistic human-AI collaboration. While generative models offer unprecedented speed in patch synthesis and risk prioritization, the “black-box” nature of these systems introduces significant risks of automation bias, where practitioners may uncritically accept flawed AI suggestions. Empirical evidence suggests that developers utilizing AI assistants often produce less secure code due to over-reliance, yet paradoxically report higher confidence in their output [6]. Consequently, effective operational governance requires robust human-in-the-loop (HITL) validation frameworks that optimize the division of labor: AI handles high-volume pattern recognition and initial draft generation, while humans retain authority over final verification and contextual judgment [132].

Calibrating trust between developers and AI tools is a critical socio-technical challenge within this collaborative framework. Trust is not static but is dynamically constructed through the transparency of AI mechanisms and the reliability of past interactions. Studies indicate that trust is significantly influenced by community signals and shared experiences within developer ecosystems, suggesting that governance models must incorporate social proof mechanisms alongside technical metrics [133]. To mitigate undue skepticism or blind faith, explainable AI (XAI) techniques are essential. Approaches that provide fine-grained interpretations, such as highlighting specific vulnerable statements or generating counterfactual explanations, enable analysts to verify the rationale behind AI decisions, thereby enhancing the actionability of automated alerts [134; 76]. However, current explanation methods often fail to align with analyst workflows, underscoring the need for human-centric interface designs that present confidence levels and causal rationales clearly [135].

Operational governance must also address the legal and ethical complexities inherent in deploying autonomous defense. The implementation of self-healing architectures and automated patching agents raises profound questions regarding liability when AI-induced repairs introduce new vulnerabilities or fail to detect critical threats [136]. The concept of “dislocated accountability” in AI supply chains complicates liability allocation, as responsibility is diffused across model developers, integrators, and end-users [87]. Emerging regulatory frameworks, such as the EU AI Act and ISO 42001, demand rigorous auditing and documentation of AI decision-making processes, yet traditional black-box audits are insufficient for verifying complex model behaviors [92; 137]. A defense-in-depth approach to governance is therefore required, integrating functional, lifecycle, and threat-based perspectives to manage frontier AI risks effectively [97].

Future research must focus on developing adaptive HITL protocols that dynamically adjust the level of human intervention based on the AI’s confidence and the criticality of the asset. Furthermore, establishing standardized benchmarks for evaluating the socio-technical efficacy of AI security tools—beyond mere accuracy metrics—is vital for fostering trustworthy adoption. By bridging the gap between algorithmic capability and human oversight, the community can realize a resilient operational defense strategy that leverages the scalability of AI while maintaining the ethical and safety guarantees provided by human expertise.

6 Evaluation Benchmarks, Explainability, and Socio-Technical Challenges

6.1 Benchmarking Rigor and Dataset Limitations in AI Security Research

The validity of AI-driven vulnerability detection research is fundamentally contingent upon the quality and representativeness of underlying datasets, yet current benchmarking practices suffer from systemic flaws that threaten the reliability of reported performance metrics. A primary concern is the pervasive issue of data leakage and temporal invalidity, where overlapping training and testing distributions inflate accuracy scores without reflecting genuine generalization capabilities. Recent empirical investigations reveal that state-of-the-art deep learning models often achieve high performance on established benchmarks like BigVul only to degrade precipitously—sometimes to the level of random guessing—when evaluated on chronologically split or deduplicated datasets that better simulate real-world deployment scenarios [59]. This phenomenon is exacerbated by severe label noise; studies indicate that 20% to 71% of vulnerability labels in prominent datasets may be inaccurate, while duplication rates can reach as high as 99%, leading models to learn superficial artifacts such as variable names rather than intrinsic semantic vulnerabilities [22; 31].

Furthermore, the scarcity of ground-truth benchmarks specifically tailored to software supply-chain attacks creates a critical disconnect between academic research and industrial reality. While datasets like [138] provide foundational records of malicious packages, they are often insufficient for training robust detectors due to their limited scale and lack of diverse attack vectors compared to the vastness of benign open-source ecosystems. Most existing evaluations rely on synthetic vulnerability injection or static analysis labels, which fail to capture the subtle, context-dependent nature of actual supply-chain compromises such as dependency confusion or trojanized pre-trained models [139; 26]. The reliance on static analyzer outputs as ground truth, as seen in datasets like D2A, introduces inherent biases where models merely replicate the false positives of the labeling tool rather than learning true vulnerability patterns [140].

The limitations extend to the evaluation metrics themselves, which predominantly prioritize classification accuracy over operational utility. High accuracy on balanced, curated datasets often masks unacceptably high false positive rates that render tools impractical for large-scale codebases, where the cost of manual triage becomes prohibitive [141]. Moreover, current benchmarks rarely assess robustness against adversarial perturbations, leaving detectors vulnerable to evasion techniques that utilize semantic-preserving code transformations [33]. Emerging efforts such as [142] and [59] attempt to mitigate these issues by introducing repository-level context and rigorous deduplication, yet a unified standard for evaluating supply-chain specific risks remains absent. Future research must transition towards dynamic, temporally valid benchmarks that incorporate verified supply-chain attack instances and adopt multi-dimensional metrics emphasizing precision, robustness, and the economic cost of validation, thereby ensuring that AI security tools are evaluated under conditions that faithfully reflect the adversarial complexity of modern software ecosystems.

6.2 Explainable AI for Trustworthy Vulnerability Localization and Remediation

While rigorous data foundations and reproducible methodologies are prerequisites for trustworthy AI, the practical adoption of deep learning (DL) models in software supply-chain security is frequently hindered by their opaque “black-box” nature, which obscures the rationale behind vulnerability alerts and impedes developer trust. To bridge this gap and address the validity concerns raised by data quality issues, Explainable AI (XAI) has emerged as a critical paradigm for transforming raw detection scores into actionable, verifiable insights. Current methodologies predominantly focus on localizing the specific code statements responsible for a vulnerability, moving

beyond coarse-grained function-level predictions to counter the tendency of models to learn superficial artifacts. For instance, [134] introduces IVDetect, which leverages Program Dependency Graphs (PDG) to isolate vulnerable statements, significantly outperforming baseline interpretation models in ranking accuracy. Similarly, [143] combines graph-based and sequence-based networks to capture both local and global contexts, achieving superior precision in statement-level localization compared to traditional static analyzers. These approaches directly address the fundamental limitation identified in [31], where models often rely on superficial artifacts like variable names rather than semantic root causes, a problem exacerbated by the noisy datasets discussed previously.

Beyond structural attribution, recent frameworks integrate causal inference and contrastive learning to mitigate spurious correlations that often plague GNN-based detectors, thereby enhancing the robustness of explanations against the dataset biases inherent in current benchmarks. [144] proposes a dual-view causal inference mechanism that enhances model robustness against random perturbations, ensuring that explanations reflect true vulnerability triggers rather than dataset biases. Furthermore, [145] demonstrates that integrating program slicing with deep learning can pinpoint vulnerability-critical lines with approximately 90% accuracy, effectively translating abstract graph embeddings into concrete code paths. This shift towards fine-grained interpretability is essential for remediation, as highlighted by [146], which shows that focusing on vulnerable statements rather than entire functions significantly improves the prediction of exploitability and severity metrics. Such precision is vital for distinguishing genuine supply-chain threats from the false positives that often arise from flawed benchmarking practices.

The rise of Large Language Models (LLMs) has introduced a new dimension to explainability through natural language rationales, offering a potential path to overcome the limitations of static analysis. Unlike static saliency maps, LLMs can generate human-readable descriptions of vulnerability root causes and potential fixes. [66] utilizes a Retrieval-Augmented Generation framework to provide high-quality explanations that have been shown to improve manual detection accuracy by human analysts. However, the reliability of these generated explanations remains a concern, echoing the broader reproducibility challenges facing the field; [58] reveals that advanced models often produce unfaithful reasoning and non-deterministic responses when code semantics are subtly altered. To address this, [62] advocates for vulnerability-semantics-guided prompting strategies that force models to articulate step-by-step logical deductions, thereby enhancing the transparency of the detection process.

Despite these advances, significant challenges persist in aligning explanatory fidelity with model performance, creating a tension that complicates the verification of defensive methodologies. There is an inherent trade-off between the complexity of deep architectures required for high-accuracy detection and the simplicity needed for human-understandable explanations. Moreover, as noted in [135], many XAI techniques are designed without sufficient consideration for the specific workflows of security analysts, leading to a disconnect between theoretical explainability and practical utility. This gap underscores the necessity for the rigorous experimental standardization discussed in the following section, as without reproducible foundations, the fidelity of these explanations remains questionable. Future research must therefore prioritize the development of interactive, human-centric explanation interfaces that not only highlight vulnerable code but also quantify confidence levels and suggest verified patches, fostering a collaborative environment where AI acts as a transparent partner rather than an autonomous oracle.

6.3 Reproducibility Crisis and Experimental Standardization in AI Security

The reproducibility crisis in AI-driven software supply-chain security represents a critical impediment to the validation of defensive methodologies and the establishment of trustworthy benchmarks. Unlike traditional software security, where deterministic static analysis yields consistent results, AI-based vulnerability detection introduces stochastic variability inherent to neural architecture training and data sampling. A primary barrier to replication is the compromised quality and availability of foundational datasets. Empirical investigations reveal that state-of-the-art vulnerability datasets suffer from severe data quality issues, including label inaccuracies ranging from 20% to 71% and duplication rates as high as 99%, which artificially inflate model performance and obscure true generalization capabilities [22]. Furthermore, the reliance on synthetic vulnerability injection or outdated patches often fails to capture the nuanced semantics of real-world supply-chain attacks, leading to models that learn dataset-specific artifacts rather than intrinsic vulnerability patterns [31; 142].

Beyond data integrity, the lack of standardized experimental configurations exacerbates the reproducibility gap. Many studies omit crucial details regarding hyperparameter tuning, random seed initialization, and the specific versions of dependency libraries, rendering exact replication impossible. This opacity is compounded by the frequent unavailability of source code and model weights due to proprietary restrictions or the sheer complexity of reconstructing distributed training environments. Consequently, claims of superior performance in vulnerability detection often collapse when subjected to rigorous, out-of-distribution testing or when evaluated against cleaner, more diverse benchmarks [57]. The situation is further complicated in the realm of generative AI, where black-box access to large language models prevents auditors from verifying the internal mechanisms driving security assessments, thereby limiting the scope of external validation to superficial output observation [92].

To mitigate these challenges, the community must transition toward rigorous experimental standardization and transparent reporting protocols. Recent efforts emphasize the necessity of decoupling a model’s reasoning capabilities from its ability to retrieve external knowledge, advocating for unified evaluation frameworks that isolate specific security competencies [147]. Additionally, the adoption of containerized environments and immutable infrastructure is essential to freeze experimental conditions, ensuring that dependency drift does not alter outcomes over time. The integration of formal verification techniques offers a complementary path, providing mathematical guarantees that can validate AI-generated fixes independent of probabilistic model behavior [74]. Future research must prioritize the creation of “living” benchmarks that evolve alongside the threat landscape, incorporating verified supply-chain attack vectors and enforcing strict artifact evaluation criteria akin to those emerging in top-tier security venues. Only through such standardized, transparent, and robust methodological frameworks can the field overcome the current reproducibility crisis and foster genuine confidence in AI-secured software supply chains.

6.4 Socio-Technical Dynamics of Developer Trust and Tool Adoption

While the previous section established that the reproducibility crisis fundamentally undermines the technical validity of AI explanations, the practical adoption of AI-driven security tools is further constrained by the complex socio-technical dynamics governing developer trust and organizational workflow integration. Trust in this context is not a binary state but a calibrated psychological construct, heavily influenced by perceived reliability, explainability, and the alignment of automated suggestions with established engineering practices. Empirical evidence suggests a paradoxical re-

lationship where developers increasingly rely on AI assistants for security-critical tasks despite harboring significant mistrust, often engaging in superficial validation that fails to detect subtle vulnerabilities introduced by the tools themselves [148]. This “automation bias” is exacerbated when tools generate plausible yet insecure code, leading developers to overestimate the security of their output compared to manual coding [6].

The formation of appropriate trust is further mediated by community signals and social proof within open-source ecosystems. Developers frequently lack direct insight into model mechanics—compounded by the opacity issues highlighted previously—relying instead on collective sense-making through online forums and peer experiences to evaluate tool efficacy [133]. However, traditional indicators of community trust, such as user reputation or answer popularity on platforms like Stack Overflow, have proven unreliable for security contexts, often failing to distinguish between secure and insecure code snippets [149]. Consequently, the propagation of insecure patterns via AI tools risks being amplified by these flawed social feedback loops. To mitigate this, recent design probes suggest that interfaces must explicitly communicate model limitations and performance metrics, allowing developers to configure expectations and validate suggestions against clear indicators of model uncertainty [150].

Organizational barriers present an equally formidable challenge to adoption, characterized by the friction between automated security recommendations and existing Continuous Integration/Continuous Deployment (CI/CD) pipelines. While AI promises to accelerate DevSecOps by automating vulnerability detection, the integration of such tools often encounters resistance due to high false positive rates, lack of actionable explanations, and the disruption of established code review workflows [151]. Static analysis tools, even when enhanced by machine learning, are frequently disregarded if they generate excessive noise or fail to prioritize warnings relevant to the specific code change, highlighting the need for context-aware prioritization mechanisms [152]. Furthermore, the “black-box” nature of deep learning models hinders adoption in high-assurance environments where developers require causal explanations for flagged vulnerabilities to perform effective remediation [153].

The path forward requires a shift from purely technical optimization to socio-technical co-design. Effective adoption strategies must incorporate human-in-the-loop validation frameworks that balance automation with expert oversight, ensuring that AI serves as a decision support system rather than an autonomous authority [136]. These socio-technical barriers set the stage for the profound legal and ethical ambiguities discussed in the following section, particularly regarding liability allocation when automated systems fail. Future research must focus on developing adaptive interfaces that calibrate trust dynamically based on real-time tool performance and developer expertise, while simultaneously establishing robust governance frameworks that define accountability for AI-induced security failures [93]. Only by addressing these human and organizational dimensions can the potential of AI in securing software supply chains be fully realized.

6.5 Legal Liability, Ethical Accountability, and Regulatory Compliance

The integration of artificial intelligence into software supply-chain security introduces profound legal and ethical ambiguities, particularly regarding liability allocation when automated systems fail. As AI-driven tools increasingly assume responsibility for vulnerability detection and patch generation, the traditional dichotomy of developer versus vendor liability becomes obscured. When an AI assistant generates insecure code or fails to identify a critical flaw, determining accountability is complicated by the “dislocated accountabilities” inherent in modular AI supply chains,

where responsibility is often diffused across data providers, model developers, and end-users [87]. Empirical studies indicate that developers often over-trust AI suggestions, leading to the adoption of vulnerable code, which raises questions about whether liability rests with the user for insufficient validation or the provider for deploying unreliable models [6]. Furthermore, the opacity of deep learning models challenges the establishment of negligence, as the inability to interpret decision-making processes hinders the proof of causality required in legal recourse [95].

Regulatory frameworks are rapidly evolving to address these systemic risks, most notably through the European Union’s AI Act, which imposes strict transparency and risk management obligations on high-risk AI systems. Compliance necessitates a shift from voluntary ethical guidelines to enforceable technical standards, requiring detailed documentation of data lineage and model limitations akin to an AI Bill of Materials (AIBOM) [85]. Existing cybersecurity governance frameworks, such as NIST CSF and ISO 27001, are currently being evaluated for their readiness to manage LLM-specific risks, with findings suggesting that while ISO 42001 offers comprehensive AI management controls, significant gaps remain in addressing the unique threat vectors of generative models [137]. The “many hands” problem in algorithmic supply chains further complicates regulatory enforcement, as cross-border data flows and distributed development practices create opportunities for regulatory arbitrage [88].

To mitigate these challenges, the security community must adapt Coordinated Vulnerability Disclosure (CVD) protocols into a specialized Coordinated Flaw Disclosure (CFD) framework tailored for AI-specific defects, such as data poisoning and backdoor triggers. Unlike traditional software bugs, AI flaws often stem from statistical anomalies or training data contamination, requiring a disclosure ecosystem that balances public safety with the protection of proprietary model weights [154]. Establishing “safe harbor” policies is critical to encourage independent red-teaming and participatory auditing without fear of legal retaliation under laws like the Computer Fraud and Abuse Act [154]. Moreover, assurance mechanisms must evolve beyond black-box testing; rigorous audits require white-box and “outside-the-box” access to training methodologies and hyperparameters to validate safety claims effectively [92]. Future research must focus on formalizing these liability frameworks and developing verifiable credentials that allow for selective disclosure of supply-chain provenance, thereby fostering an ecosystem where trust is earned through transparent, auditable, and legally accountable engineering practices [93].

7 Conclusion

This subsection synthesizes the dualistic role of artificial intelligence within the software supply chain, where AI serves simultaneously as a potent defensive mechanism and a critical attack vector. Our analysis reveals that while deep learning architectures, particularly Graph Neural Networks (GNNs) and Large Language Models (LLMs), have advanced vulnerability detection beyond traditional static analysis, significant gaps remain in their robustness and generalizability. Approaches like ReGVD [46] and SySeVR [3] demonstrate efficacy in capturing semantic code properties; however, empirical studies indicate that performance often degrades precipitously in real-world scenarios due to data leakage and unrealistic training distributions [31]. Furthermore, the reliance on LLMs for zero-shot detection introduces non-deterministic reasoning and susceptibility to adversarial perturbations, where minor syntactic changes can lead to incorrect security assessments [58; 68].

The survey highlights a critical paradox: the same generative capabilities that enable automated patch synthesis and remediation [106; 70] also exacerbate supply chain risks through the propagation

of insecure coding patterns and the facilitation of poisoning attacks [15; 6]. The integration of AI into CI/CD pipelines promises enhanced agility [151], yet it expands the attack surface to include model artifacts and training data, necessitating rigorous provenance tracking via Software Bill of Materials (SBOM) extensions [155; 156]. Current defenses often lack the adaptability required to counter sophisticated threats such as clean-label poisoning and transitive dependency hijacking [1; 138], revealing a pressing need for defense-in-depth strategies that incorporate formal verification [74].

Looking forward, the research community must prioritize the development of unified ontologies and robust, leakage-free benchmarks to accurately evaluate AI security tools [57]. Future work should focus on neuro-symbolic hybrids that combine the scalability of deep learning with the mathematical guarantees of formal methods to reduce false positives and enhance explainability [157; 62]. Additionally, establishing standards for white-box auditing and cryptographic attestation is essential to ensure model integrity and accountability across complex, multi-stakeholder supply chains [92; 87]. Ultimately, achieving trustworthy AI ecosystems requires a paradigm shift from reactive detection to proactive, security-by-design principles that address both the technical vulnerabilities of AI models and the socio-technical dynamics of their deployment [8; 14].

References

- [1] Taxonomy of Attacks on Open-Source Software Supply Chains
- [2] Adversarial Machine Learning – Industry Perspectives
- [3] SySeVR A Framework for Using Deep Learning to Detect Software Vulnerabilities
- [4] An Empirical Study of Malicious Code In PyPI Ecosystem
- [5] What do we know about Hugging Face? A systematic literature review and quantitative validation of qualitative claims
- [6] Do Users Write More Insecure Code with AI Assistants
- [7] Lost at C A User Study on the Security Implications of Large Language Model Code Assistants
- [8] Towards the Science of Security and Privacy in Machine Learning
- [9] We Have a Package for You! A Comprehensive Analysis of Package Hallucinations by Code Generating LLMs
- [10] Demystifying the Vulnerability Propagation and Its Evolution via Dependency Trees in the NPM Ecosystem
- [11] On the Effect of Transitivity and Granularity on Vulnerability Propagation in the Maven Ecosystem
- [12] Security Evaluation of Support Vector Machines in Adversarial Environments
- [13] Poisoning Attacks and Defenses on Artificial Intelligence A Survey
- [14] Trustworthy and Synergistic Artificial Intelligence for Software Engineering Vision and Roadmaps
- [15] You Autocomplete Me Poisoning Vulnerabilities in Neural Code Completion
- [16] TrojanPuzzle Covertly Poisoning Code-Suggestion Models

- [17] An LLM-Assisted Easy-to-Trigger Backdoor Attack on Code Completion Models: Injecting Disguised Vulnerabilities against Strong Detection
- [18] Poisoning Programs by Un-Repairing Code Security Concerns of AI-generated Code
- [19] Backdoors in Neural Models of Source Code
- [20] Stealthy Backdoor Attack for Code Models
- [21] Measuring Impacts of Poisoning on Model Parameters and Neuron Activations A Case Study of Poisoning CodeBERT
- [22] Data Quality for Software Vulnerability Datasets
- [23] Top Score on the Wrong Exam: On Benchmarking in Machine Learning for Vulnerability Detection
- [24] Handcrafted Backdoors in Deep Neural Networks
- [25] Towards Practical Deployment-Stage Backdoor Attack on Deep Neural Networks
- [26] Do You Trust Your Model Emerging Malware Threats in the Deep Learning Ecosystem
- [27] Asleep at the Keyboard Assessing the Security of GitHub Copilot’s Code Contributions
- [28] Generative AI and Large Language Models for Cyber Security: All Insights You Need
- [29] Natural Attack for Pre-trained Models of Code
- [30] A Search-Based Testing Framework for Deep Neural Networks of Source Code Embedding
- [31] Deep Learning based Vulnerability Detection Are We There Yet
- [32] AttackGNN Red-Teaming GNNs in Hardware Security Using Reinforcement Learning
- [33] Evasion Attacks against Machine Learning at Test Time
- [34] Robust Machine Learning Systems Challenges, Current Trends, Perspectives, and the Road Ahead
- [35] An Adversarial Approach for Explainable AI in Intrusion Detection Systems
- [36] On the Impact of Security Vulnerabilities in the npm and RubyGems Dependency Networks
- [37] SBOM.EXE: Countering Dynamic Code Injection based on Software Bill of Materials in Java
- [38] What are Weak Links in the npm Supply Chain
- [39] Vulnerabilities in AI Code Generators Exploring Targeted Data Poisoning Attacks
- [40] On the critical path to implant backdoors and the effectiveness of potential mitigation techniques Early learnings from XZ
- [41] A Comparative Study of Vulnerability Reporting by Software Composition Analysis Tools
- [42] SIGL Securing Software Installations Through Deep Graph Learning
- [43] Malicious Package Detection using Metadata Information
- [44] Learning to Represent Programs with Graphs

- [45] Design Effective Vulnerability Identification by Learning Comprehensive Program Semantics via Graph Neural Networks
- [46] ReGVD Revisiting Graph Neural Networks for Vulnerability Detection
- [47] An Unbiased Transformer Source Code Learning with Semantic Vulnerability Graph
- [48] LineVD Statement-level Vulnerability Detection using Graph Neural Networks
- [49] VulDeeLocator A Deep Learning-based Fine-grained Vulnerability Detector
- [50] Neural Network-based Graph Embedding for Cross-Platform Binary Code Similarity Detection
- [51] Semantic Learning and Emulation Based Cross-platform Binary Vulnerability Seeker
- [52] MANDO Multi-Level Heterogeneous Graph Embeddings for Fine-Grained Detection of Smart Contract Vulnerabilities
- [53] Vulnerability Detection with Graph Simplification and Enhanced Graph Representation Learning
- [54] Meta-Path Based Attentional Graph Learning Model for Vulnerability Detection
- [55] Limits of Machine Learning for Automatic Vulnerability Detection
- [56] Transformer-Based Language Models for Software Vulnerability Detection
- [57] DiverseVul A New Vulnerable Source Code Dataset for Deep Learning Based Vulnerability Detection
- [58] LLMs Cannot Reliably Identify and Reason About Security Vulnerabilities (Yet) A Comprehensive Evaluation, Framework, and Benchmarks
- [59] Vulnerability Detection with Code Language Models How Far Are We
- [60] Prompt-Enhanced Software Vulnerability Detection Using ChatGPT
- [61] A Preliminary Study on Using Large Language Models in Software Pentesting
- [62] Chain-of-Thought Prompting of Large Language Models for Discovering and Fixing Software Vulnerabilities
- [63] Large Language Model for Vulnerability Detection Emerging Results and Future Directions
- [64] Comparison of Static Application Security Testing Tools and Large Language Models for Repo-level Vulnerability Detection
- [65] On the Effectiveness of Function-Level Vulnerability Detectors for Inter-Procedural Vulnerabilities
- [66] Vul-RAG: Enhancing LLM-based Vulnerability Detection via Knowledge-level RAG
- [67] DLAP: A Deep Learning Augmented Large Language Model Prompting Framework for Software Vulnerability Detection
- [68] Understanding the Effectiveness of Large Language Models in Detecting Security Vulnerabilities
- [69] A Comprehensive Study of the Capabilities of Large Language Models for Vulnerability Detection

- [70] A New Era in Software Security Towards Self-Healing Software via Large Language Models and Formal Verification
- [71] How Far Have We Gone in Vulnerability Detection Using Large Language Models
- [72] sCompile Critical Path Identification and Analysis for Smart Contracts
- [73] Efficiently Detecting Reentrancy Vulnerabilities in Complex Smart Contracts
- [74] The FormAI Dataset Generative AI in Software Security Through the Lens of Formal Verification
- [75] 3DGen AI-Assisted Generation of Provably Correct Binary Format Parsers
- [76] Graph Neural Networks for Vulnerability Detection: A Counterfactual Explanation
- [77] Beyond Fidelity Explaining Vulnerability Localization of Learning-based Detectors
- [78] Combining Graph Neural Networks with Expert Knowledge for Smart Contract Vulnerability Detection
- [79] Eth2Vec Learning Contract-Wide Code Representations for Vulnerability Detection on Ethereum Smart Contracts
- [80] GPTScan Detecting Logic Vulnerabilities in Smart Contracts by Combining GPT with Program Analysis
- [81] Large Language Model-Powered Smart Contract Vulnerability Detection New Perspectives
- [82] Not all bytes are equal Neural byte sieve for fuzzing
- [83] Purple Llama CyberSecEval A Secure Coding Benchmark for Language Models
- [84] All Artificial, Less Intelligence GenAI through the Lens of Formal Verification
- [85] Trust in Software Supply Chains Blockchain-Enabled SBOM and the AIBOM Future
- [86] AuditMAI: Towards An Infrastructure for Continuous AI Auditing
- [87] Dislocated Accountabilities in the AI Supply Chain Modularity and Developers' Notions of Responsibility
- [88] Understanding accountability in algorithmic supply chains
- [89] Navigating the EU AI Act A Methodological Approach to Compliance for Safety-critical Products
- [90] Resilient Watermarking for LLM-Generated Codes
- [91] Balancing Transparency and Risk The Security and Privacy Risks of Open-Source Machine Learning Models
- [92] Black-Box Access is Insufficient for Rigorous AI Audits
- [93] Filling gaps in trustworthy development of AI
- [94] BadNets Identifying Vulnerabilities in the Machine Learning Model Supply Chain
- [95] Secure Deep Learning Engineering A Software Quality Assurance Perspective

- [96] The Pipeline for the Continuous Development of Artificial Intelligence Models – Current State of Research and Practice
- [97] Adapting cybersecurity frameworks to manage frontier AI risks: A defense-in-depth approach
- [98] DONAPI Malicious NPM Packages Detector using Behavior Sequence Knowledge Mapping
- [99] Large Language Models for Code Security Hardening and Adversarial Testing
- [100] CYBERSECEVAL 3: Advancing the Evaluation of Cybersecurity Risks and Capabilities in Large Language Models
- [101] A System for Automated Open-Source Threat Intelligence Gathering and Management
- [102] Yes, Machine Learning Can Be More Secure! A Case Study on Android Malware Detection
- [103] UNICORN Runtime Provenance-Based Detector for Advanced Persistent Threats
- [104] Security and Privacy for Artificial Intelligence Opportunities and Challenges
- [105] A Survey of Learning-based Automated Program Repair
- [106] How Effective Are Neural Networks for Fixing Security Vulnerabilities
- [107] An Exploratory Study on Fine-Tuning Large Language Models for Secure Code Generation
- [108] NAVRepair: Node-type Aware C/C++ Code Vulnerability Repair
- [109] RAP-Gen Retrieval-Augmented Patch Generation with CodeT5 for Automatic Program Repair
- [110] Multi-LLM Collaboration + Data-Centric Innovation = 2x Better Vulnerability Repair
- [111] LLM-Powered Code Vulnerability Repair with Reinforcement Learning and Semantic Reward
- [112] Examining Zero-Shot Vulnerability Repair with Large Language Models
- [113] A Look at the Time Delays in CVSS Vulnerability Scoring
- [114] Enhancing Vulnerability Prioritization Data-Driven Exploit Predictions with Community-Driven Insights
- [115] Predicting Exploitation of Disclosed Software Vulnerabilities Using Open-source Data
- [116] A Relevance Model for Threat-Centric Ranking of Cybersecurity Vulnerabilities
- [117] Linking Common Vulnerabilities and Exposures to the MITRE ATT&CK Framework A Self-Distillation Approach
- [118] Automated Software Vulnerability Assessment with Concept Drift
- [119] Learning from what we know How to perform vulnerability prediction using noisy historical data
- [120] Economic Factors of Vulnerability Trade and Exploitation
- [121] CVSS-BERT Explainable Natural Language Processing to Determine the Severity of a Computer Security Vulnerability from its Description
- [122] Will AI Make Cyber Swords or Shields A few mathematical models of technological progress

- [123] DeepAID Interpreting and Improving Deep Learning-based Anomaly Detection in Security Applications
- [124] Manifold for Machine Learning Assurance
- [125] A False Sense of Security Revisiting the State of Machine Learning-Based Industrial Intrusion Detection
- [126] DETERRENT Detecting Trojans using Reinforcement Learning
- [127] Combining ensemble methods and social network metrics for improving accuracy of OCSVM on intrusion detection in SCADA systems
- [128] Deep Reinforcement Fuzzing
- [129] Simulating SQL Injection Vulnerability Exploitation Using Q-Learning Reinforcement Learning Agents
- [130] Sensor networks security based on sensitive robots agents. A conceptual model
- [131] Security of AI Agents
- [132] Rise of the HaCRS Augmenting Autonomous Cyber Reasoning Systems with Human Assistance
- [133] It would work for me too How Online Communities Shape Software Developers' Trust in AI-Powered Code Generation Tools
- [134] Vulnerability Detection with Fine-grained Interpretations
- [135] SoK Explainable Machine Learning for Computer Security Applications
- [136] Trust Enhancement Issues in Program Repair
- [137] From COBIT to ISO 42001 Evaluating Cybersecurity Frameworks for Opportunities, Risks, and Regulatory Compliance in Commercializing Large Language Models
- [138] Backstabber's Knife Collection A Review of Open Source Software Supply Chain Attacks
- [139] Towards Measuring Supply Chain Attacks on Package Managers for Interpreted Languages
- [140] D2A A Dataset Built for AI-Based Vulnerability Detection Methods Using Differential Analysis
- [141] Automated software vulnerability detection with machine learning
- [142] ReposVul A Repository-Level High-Quality Vulnerability Dataset
- [143] VELVET a noVel Ensemble Learning approach to automatically locate Vulnerable Statements
- [144] Coca Improving and Explaining Graph Neural Network-Based Vulnerability Detection Systems
- [145] The Vulnerability Is in the Details Locating Fine-grained Information of Vulnerable Code Identified by Graph-based Detectors
- [146] On the Use of Fine-grained Vulnerable Code Statements for Software Vulnerability Assessment Models

- [147] LLM4Vuln A Unified Evaluation Framework for Decoupling and Enhancing LLMs' Vulnerability Reasoning
- [148] Using AI Assistants in Software Development: A Qualitative Study on Security Practices and Concerns
- [149] How Reliable is the Crowdsourced Knowledge of Security Implementation
- [150] Investigating and Designing for Trust in AI-powered Code Generation Tools
- [151] AI for DevSecOps A Landscape and Future Opportunities
- [152] An Empirical Study of Static Analysis Tools for Secure Code Review
- [153] Explainable AI for Software Engineering
- [154] Coordinated Disclosure for AI Beyond Security Vulnerabilities
- [155] An Empirical Study on Software Bill of Materials Where We Stand and the Road Ahead
- [156] Challenges of Producing Software Bill Of Materials for Java
- [157] LLM-Assisted Static Analysis for Detecting Security Vulnerabilities